

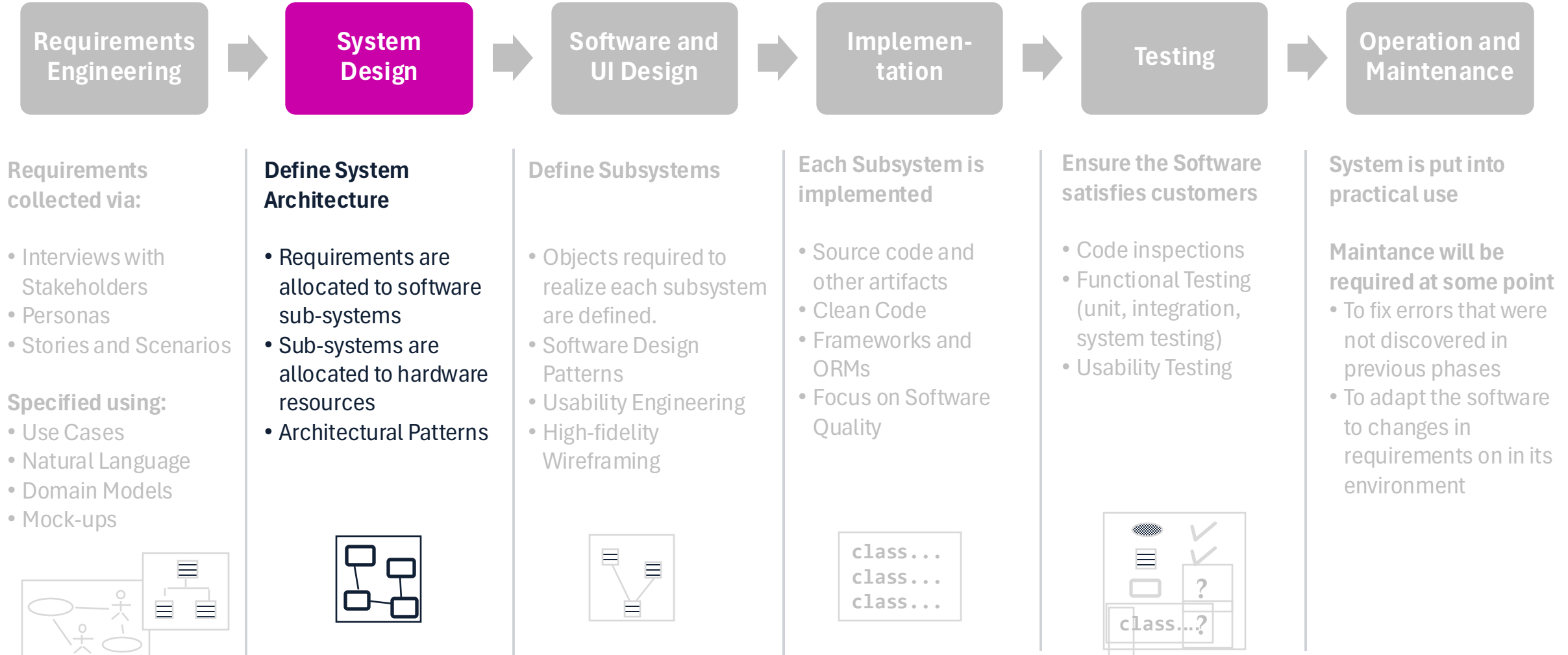
UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
SOFTWARE ENGINEERING – LECTURE 13

System Design and Software Architectures

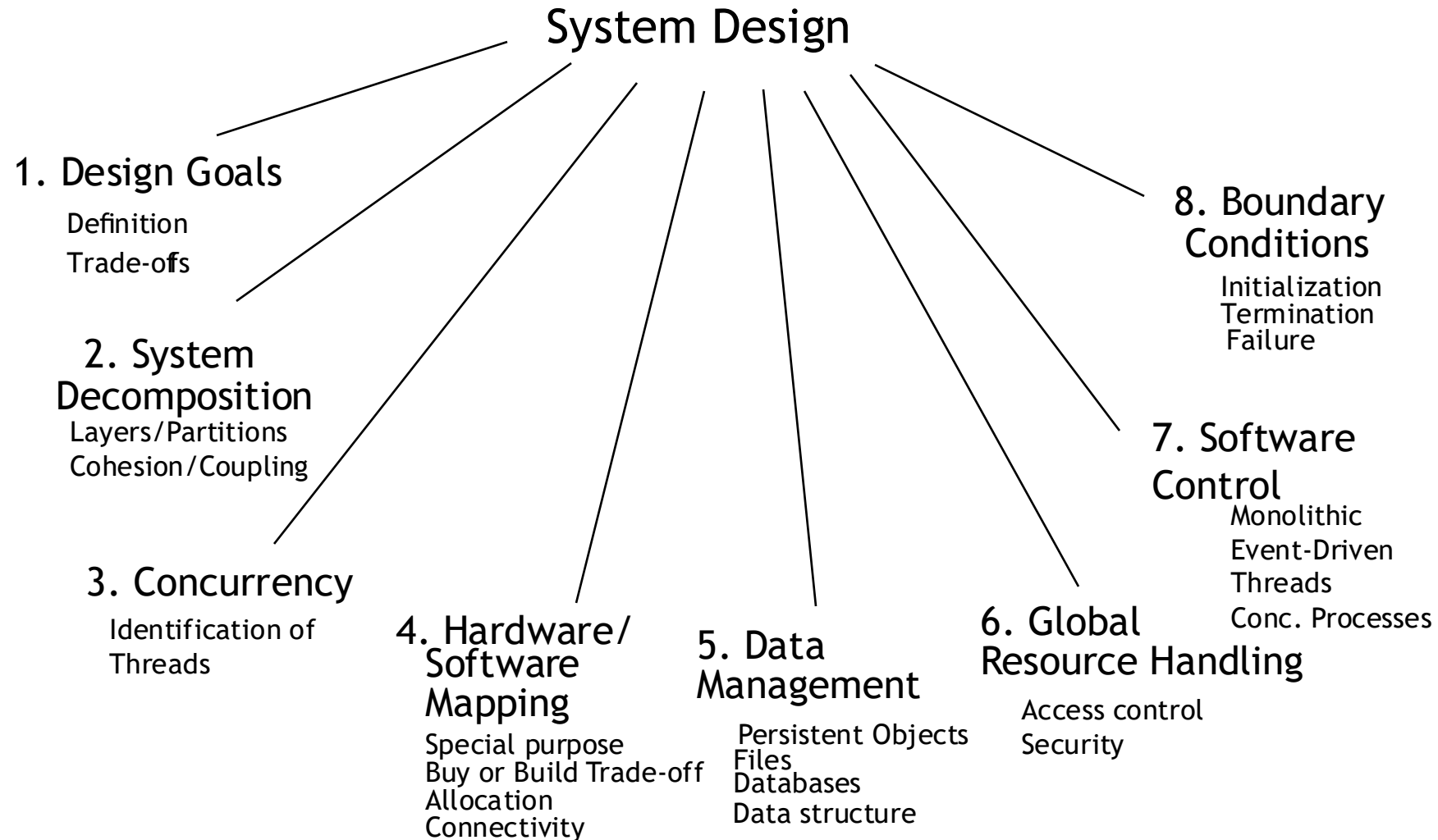
Prof. Bernardo Breve



Software Development Life Cycle



System Design steps

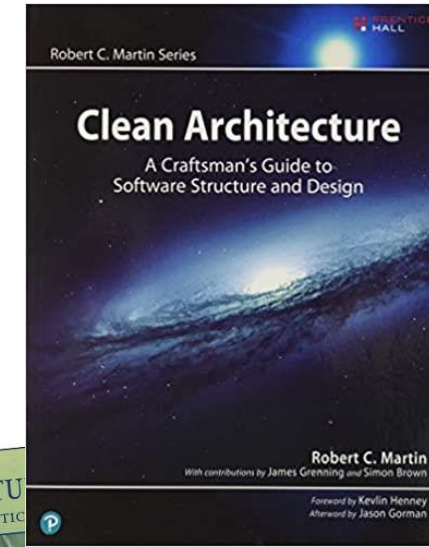
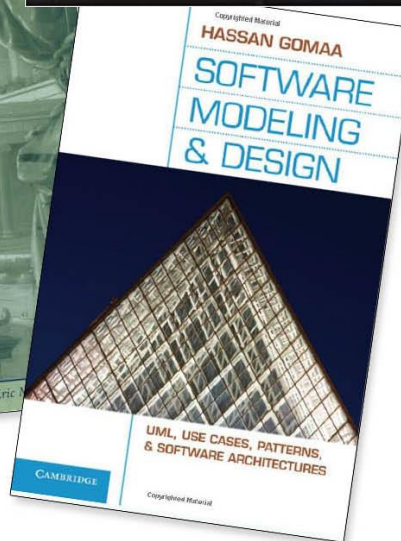
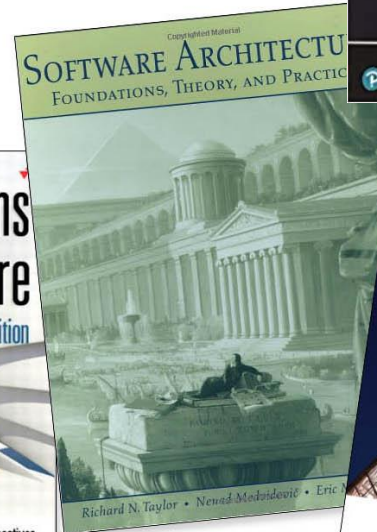
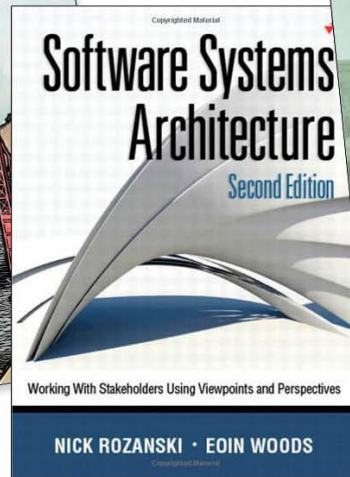
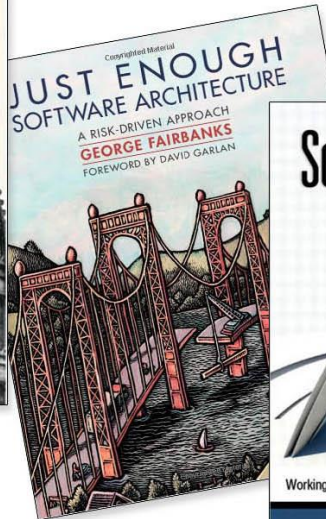
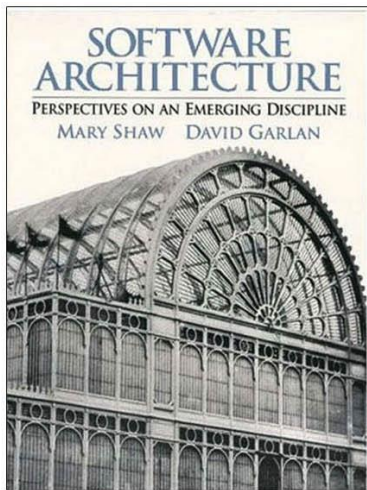


Organizing Subsystems into Architectures



Software Architecture

- Key part of Software Engineering



What Is Software Architecture?

- Software architecture is the structure (or structures) of the system, which comprise software elements, the externally visible properties of these elements, and the relationships among them.
- The exact structures to consider and the ways to represent them vary according to engineering goals.



Implications of this Definition – 1

- A software architecture is an abstraction of a system.
 - Architecture defines elements and how they interact.
 - Architecture suppresses purely local information about elements; private details are not architectural.
- Externally-visible properties of elements are assumptions that one elements can make about another:
 - provided services, required services, performance characteristics, fault handling, resource usage

Implications of this Definition – 2

- **Every system has an architecture.**
 - Every system is composed of elements and there are relationships among them.
 - In the simplest case, a system is composed of a single element, related only to itself.
- Just having an architecture is different from having an architecture that is known to everyone:
 - The architecture versus specification of the architecture
- If you don't explicitly develop an architecture, you will get one anyway – and you might not like what you get.

Implications of this Definition – 3

- This means that box-and-line drawings alone are *not* architectures; but they are just a starting point.
 - You might *imagine* the behavior of a box labeled “database” or “executive” -- but that’s all
 - You need to add specifications and properties.
- Systems have many structures (views).
 - No single structure can be *the* architecture.
 - The set of candidate structures is not fixed or prescribed: choose whatever is useful for analysis, communication, or understanding.

Architectures

- The architecture of a software system is defined in the first phase of System Design (Architectural design).
- The primary purpose is to break down the system into subsystems:
 - The creation of several distinct components is less complex than the creation of a monolithic system.
 - It allows for parallel development.
 - It promotes modifiability, reusability, portability, etc.

Architectures

- Defining an architecture means mapping functionality onto modules.
 - E.g.: User interface module, database access module, security management module, etc.
- The definition of architectures must also follow the concepts of high cohesion and low coupling.
 - Each subsection of the architecture must provide highly related services, seeking to limit the number of other modules with which it is linked.

Definition of architecture

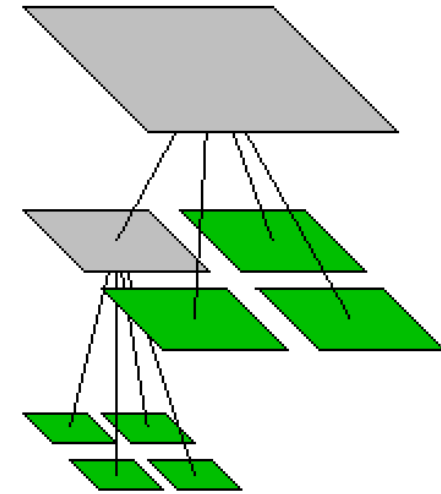
- The definition of architecture is usually done from two different points of view, which lead to the final solution:
 - Identification and relationship of subsystems
 - Definition of control policies
- Both choices are crucial:
 - it is difficult to change them once development has started, as many subsystem interfaces would have to be changed.

Subsystem identification



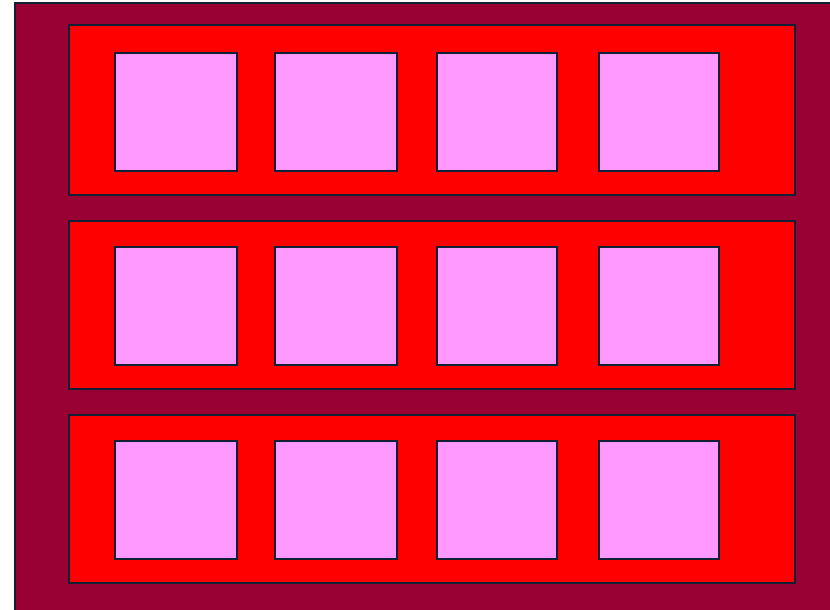
Design Principle: Divide and conquer

- Trying to deal with something big all at once is harder than dealing with a set of smaller things
 - Each individual component is smaller, and therefore easier to understand
 - Parts can be replaced or changed without having to replace or extensively change other parts.
 - Separate people can work on separate parts
 - An individual software engineer can specialize

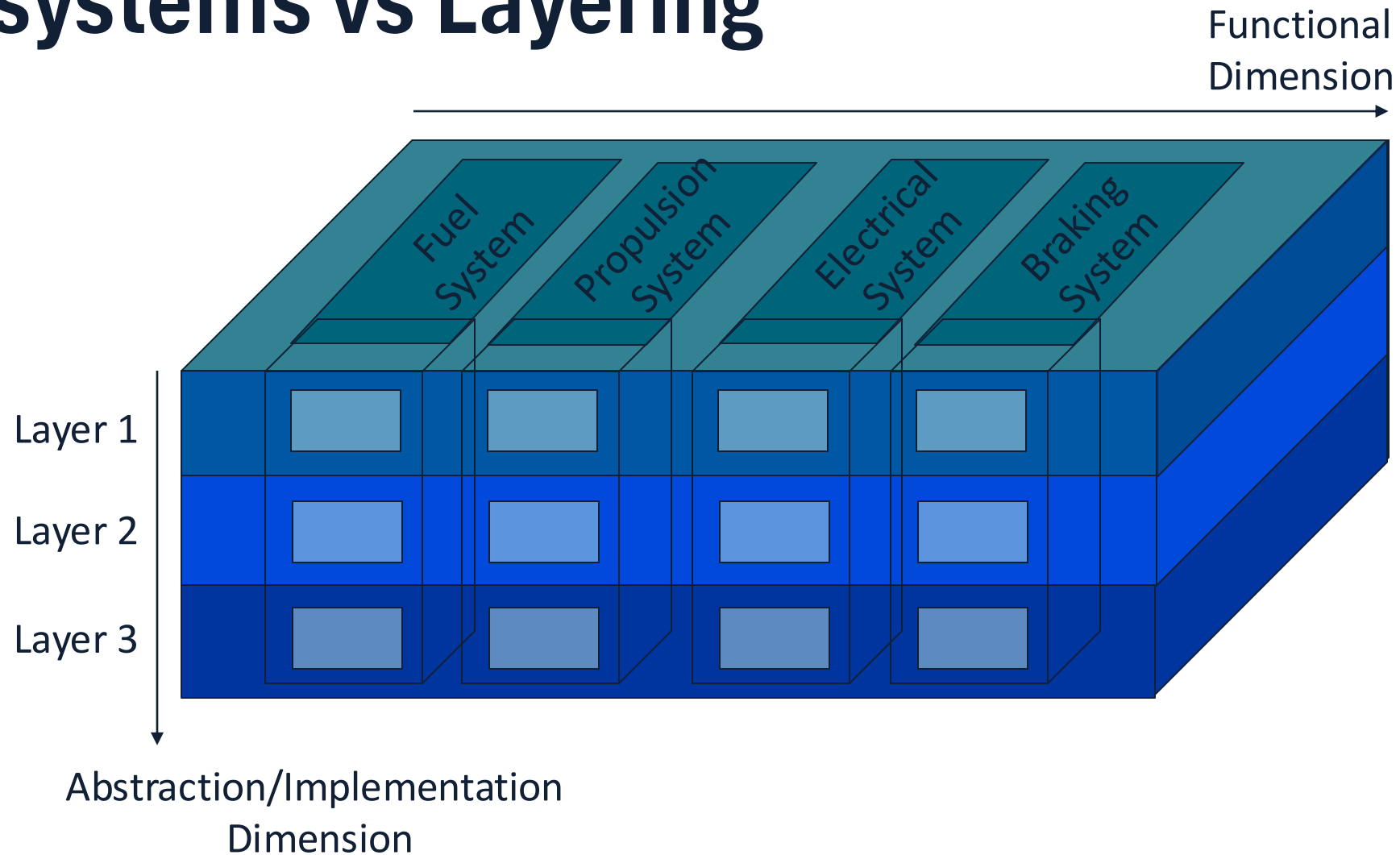


Ways of dividing a software system

- A system is divided up into
 - Layers & subsystems
 - A subsystem can be divided up into one or more packages
 - A package is divided up into classes
 - A class is divided up into methods



Subsystems vs Layering

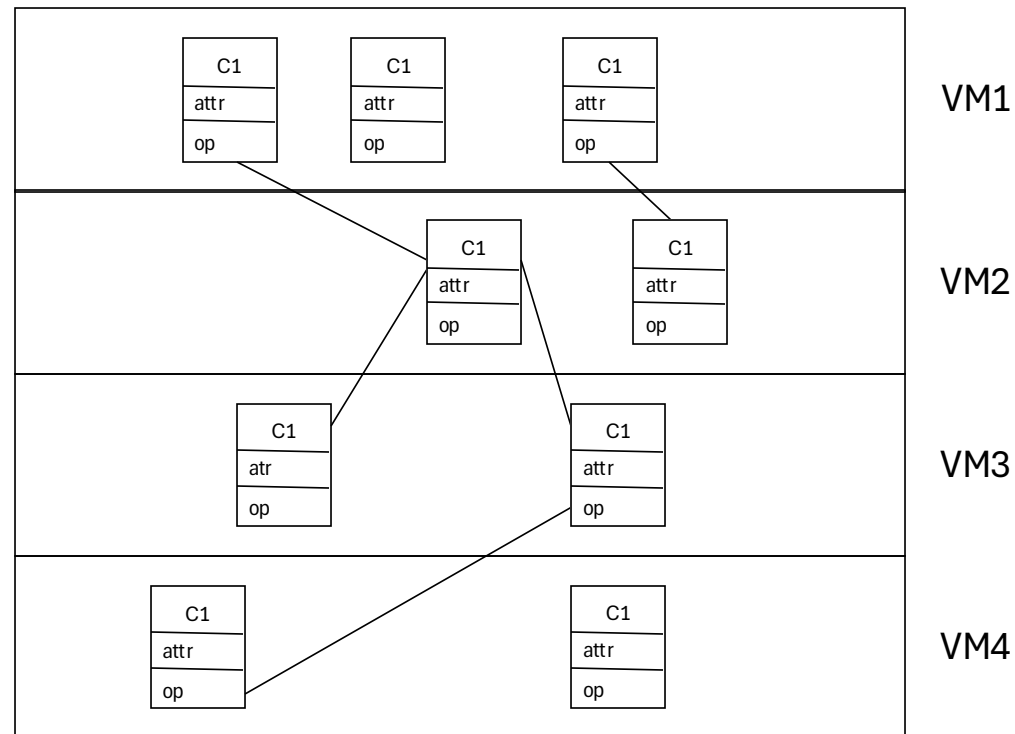


Layers

- A hierarchical decomposition of a system consists of an ordered set of layers.
 - A layer is a grouping of subsystems that provide related services, possibly implemented using services from other layers.
 - A layer can only depend on lower-level layers.
 - A layer has no knowledge of higher-level layers.
- Closed architecture: each layer can only access the layer immediately below it.
- Open architecture: a layer can also access lower-level layers.

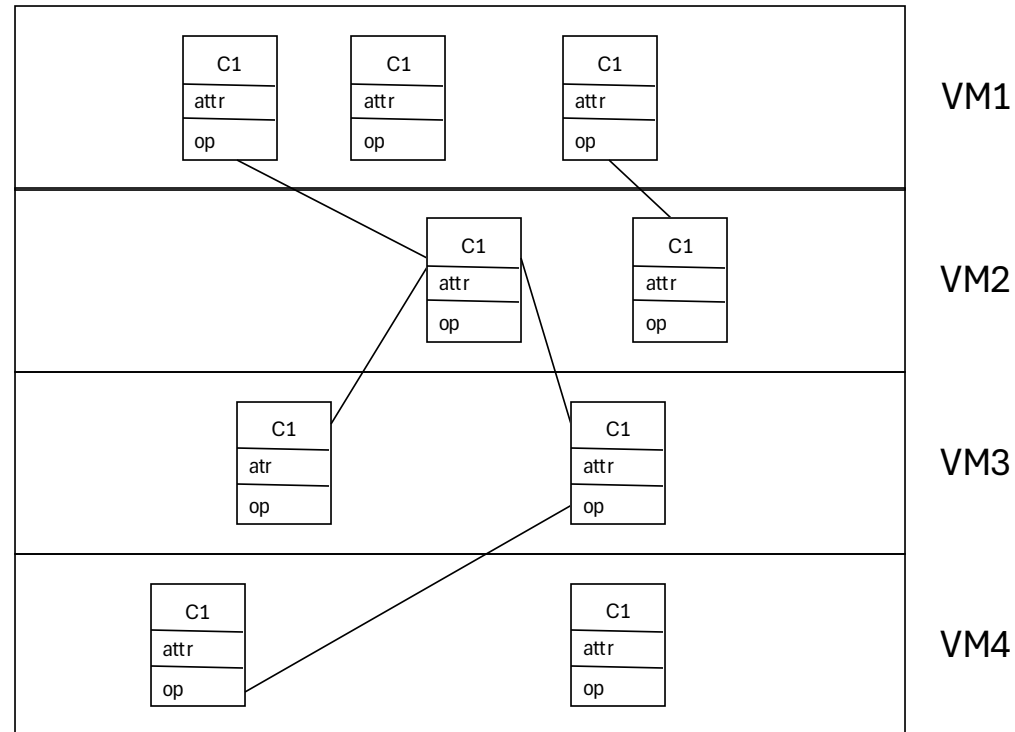
Virtual Machine (Dijkstra, 1965)

- First formalization of layered architecture
- A system should be developed as a set of virtual machines, each built on top of the one below it.

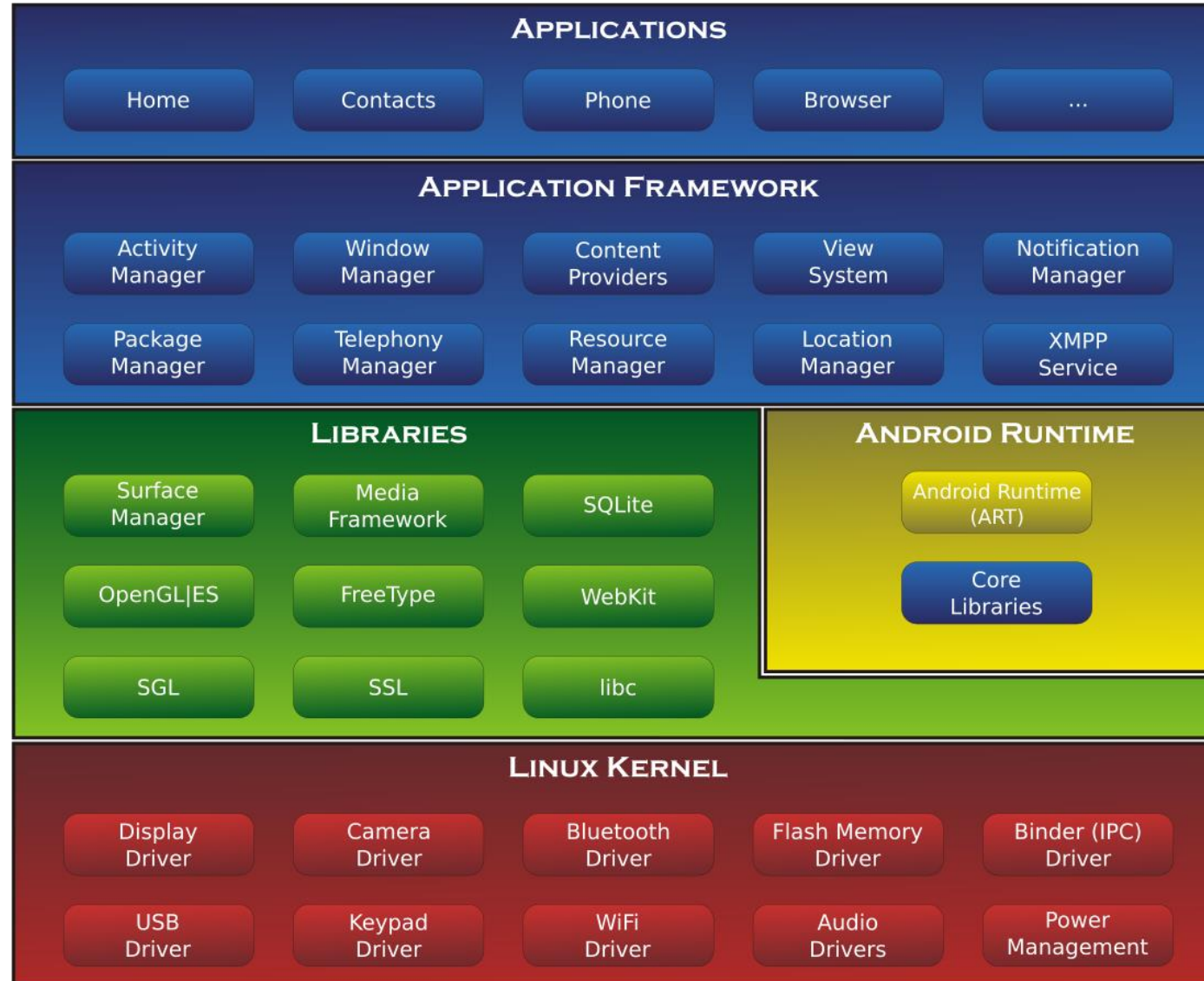


Closed Architecture

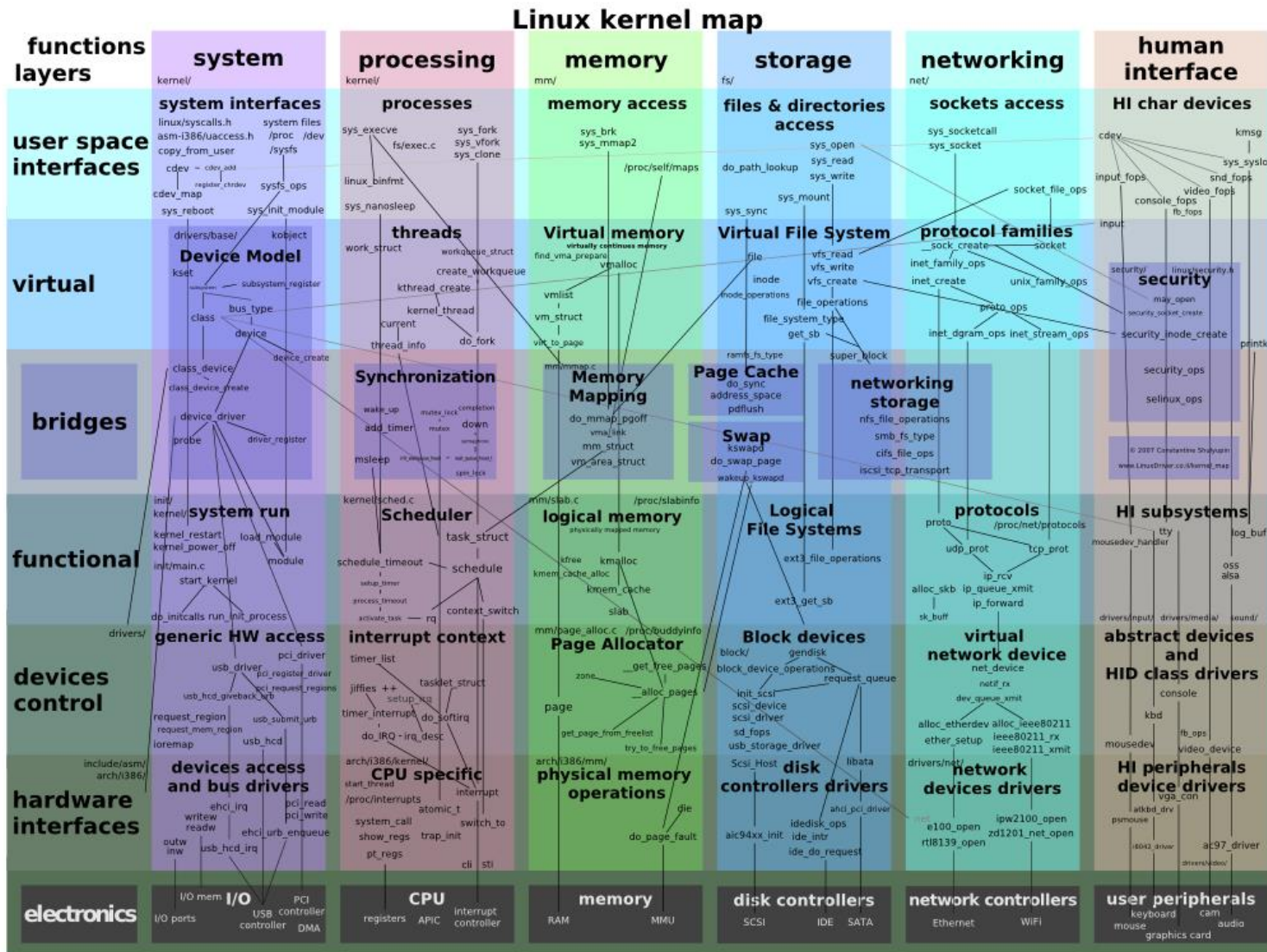
- A virtual machine can only call operations from the underlying layer
- Design goal: high maintainability and portability



Example of closed architecture: Android

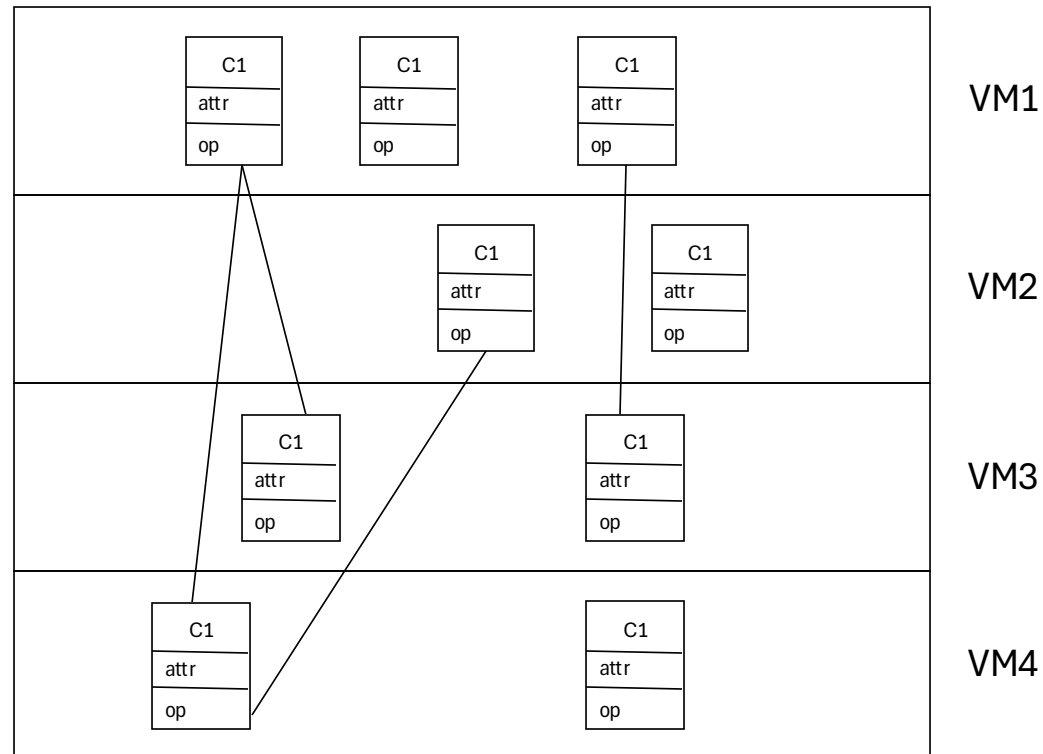


Consistency in two dimensions

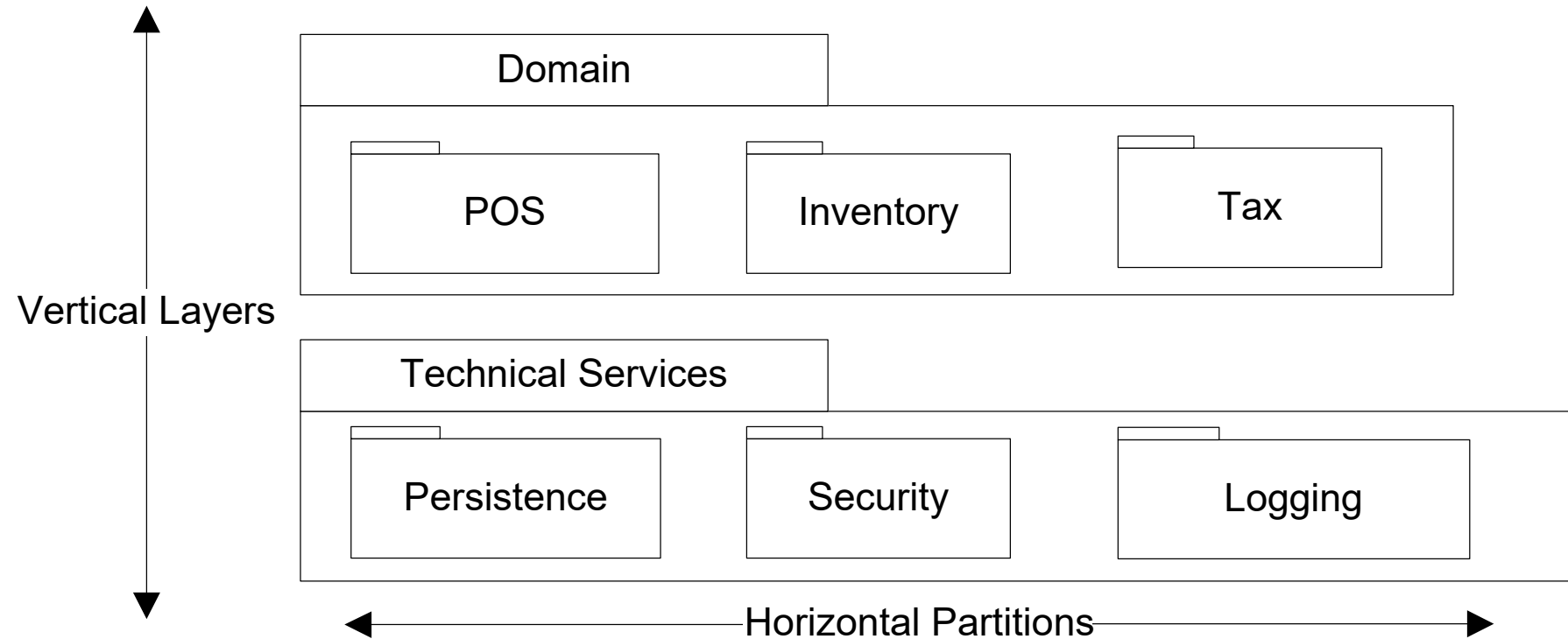


Open architecture

- A virtual machine can use the services of the machines in the underlying layers
- Design goal: runtime efficiency



Logical Architecture: Layers and Partitions



Main Architectures

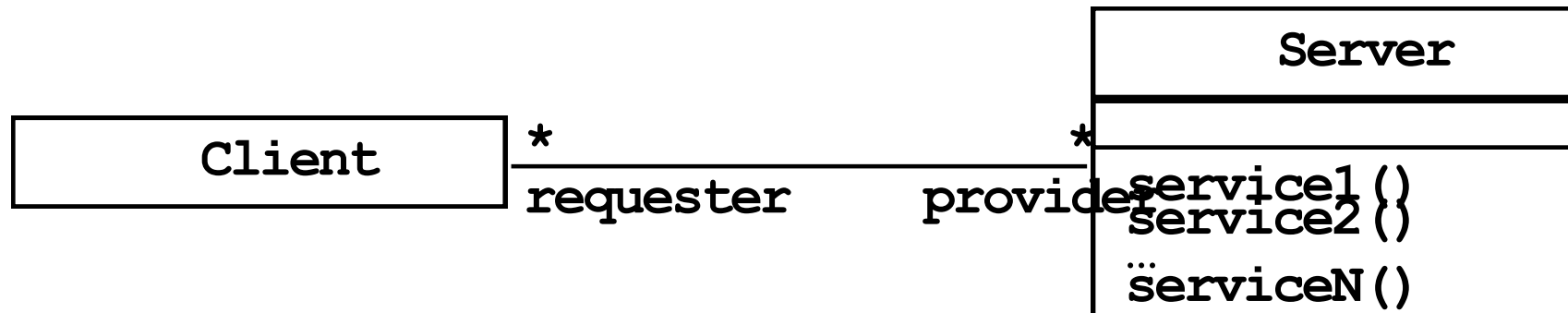


Client/Server Architecture

- It is a distributed architecture where data and processing are distributed across a network of two types of nodes:
 - Servers are powerful, dedicated processors: they offer specific services such as printing, file system management, compilation, network traffic management, and computing.
 - Clients are less powerful machines on which user applications run, using the services of the servers.
- Clients must know the names and nature of the servers;
- Servers must not know the identity and number of clients.

Client/Server Architecture

- A subsystem, called a server, provides services to instances of other subsystems called clients, which are responsible for interacting with the user.
- Clients call the server, which performs certain services and returns the result.
 - Clients know the server interface (its services).
 - Servers do not know the client interfaces.
 - The response is generally immediate.
- Users interact only with the client.

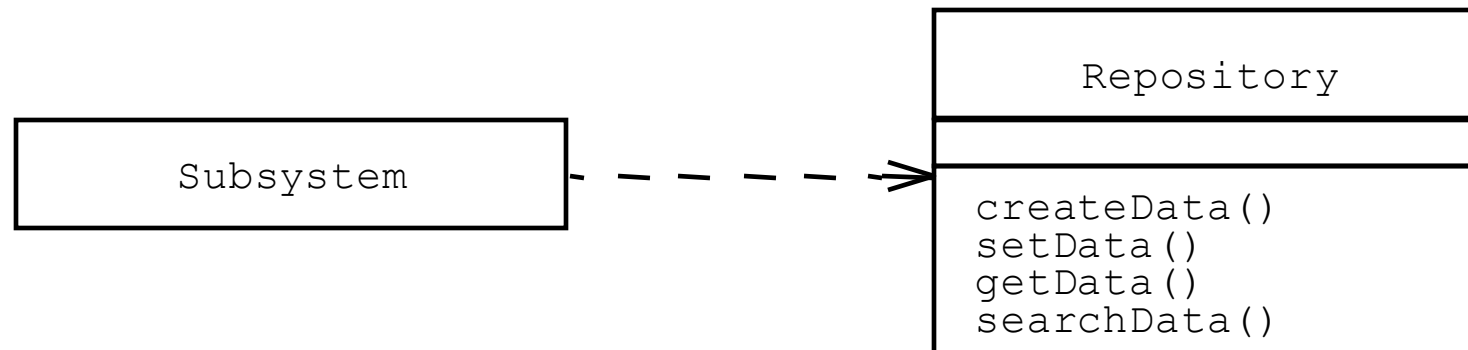


Main Software Architectures

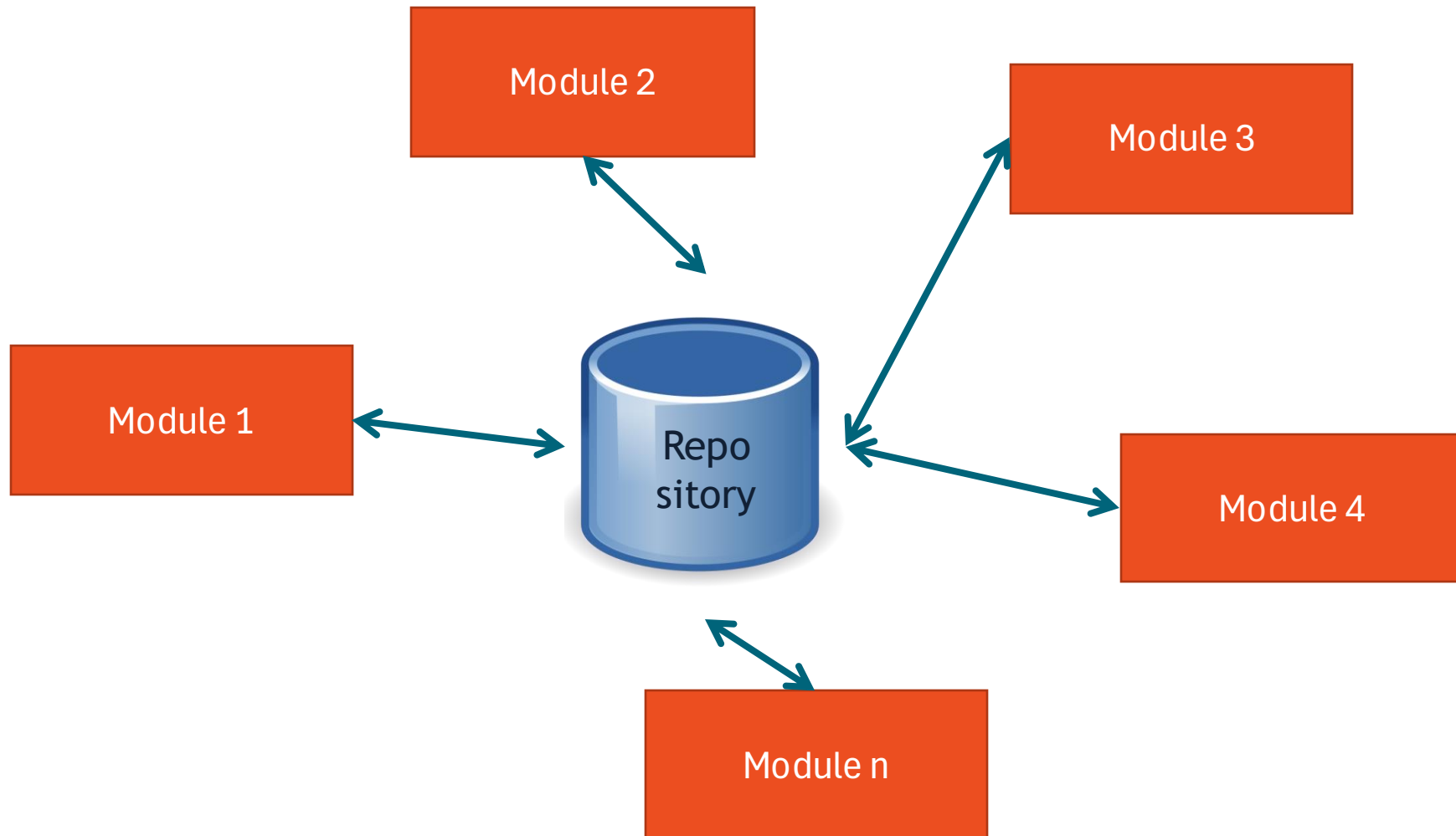
- In software engineering, various architectural styles have been defined that can be used as a basis for developing a specific software system:
 - Client/Server Architecture
 - Repository Architecture
 - Peer-To-Peer Architecture
 - Model/View/Controller

Repository Architecture

- Subsystems access and modify a single data structure called a repository.
- Subsystems are “relatively independent” (they interact only through the repository).
- Control flow is dictated either by the repository (a change in stored data) or by the subsystems (independent control flow).

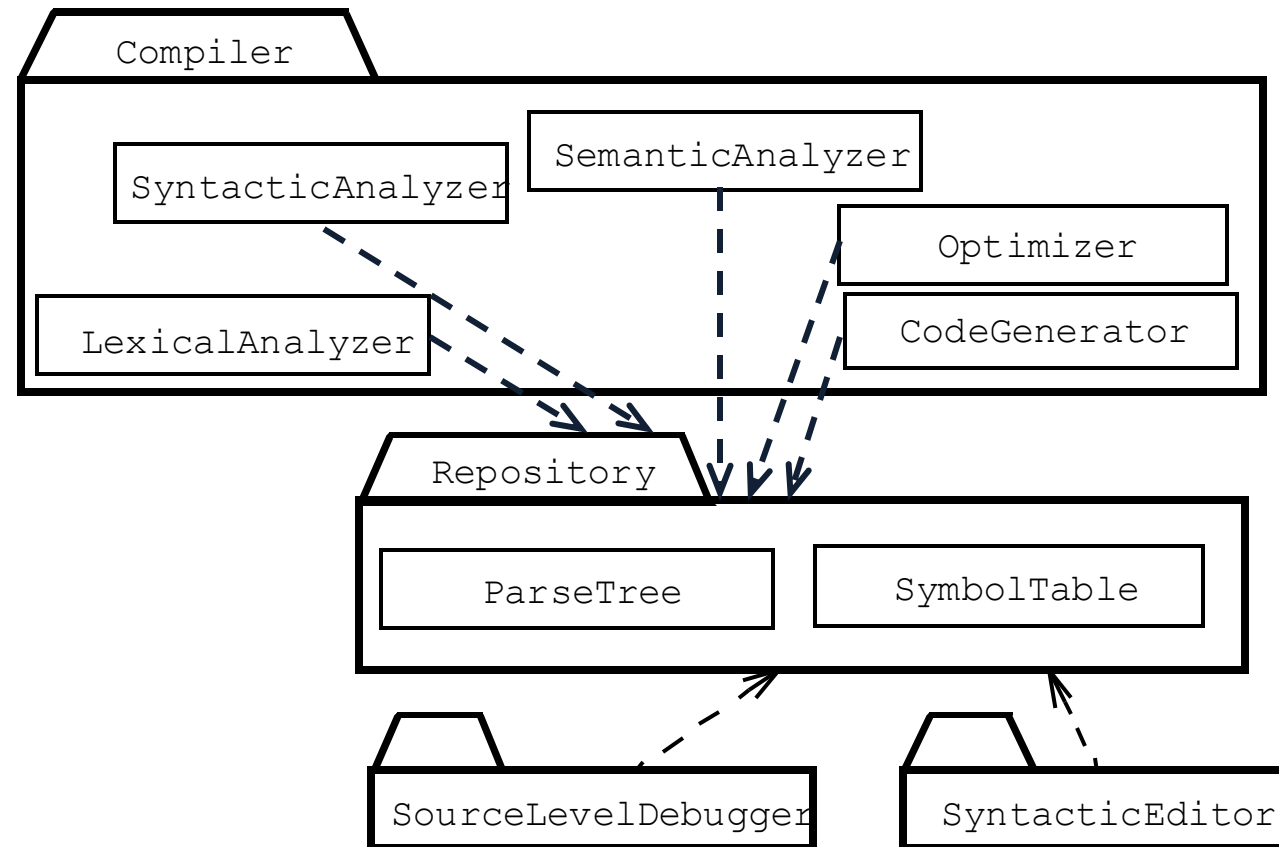


Repository Architecture



Example of Repository Architecture

- Database Management Systems
- Modern Compilers



Benefits of Repository Architecture

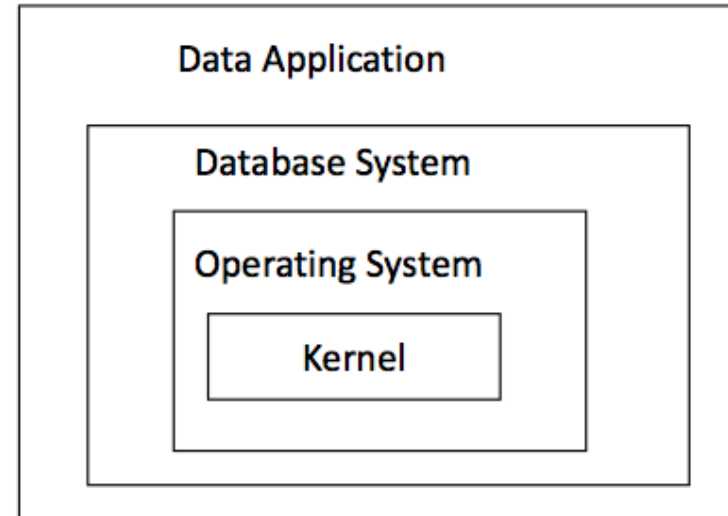
- Efficient way to share large amounts of data: write once for all to read.
- A subsystem does not have to worry about how data is produced/used by any other subsystem.
- Centralized management of backup, security, access control, error recovery, etc.
- The data sharing model is published as a repository schema → easy to add new subsystems.

Drawbacks of Repository Architecture

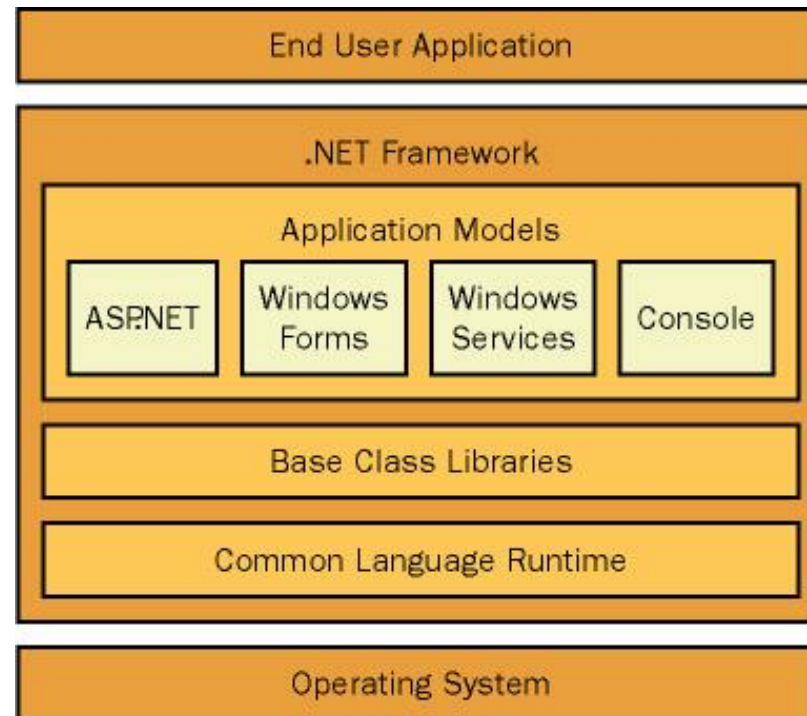
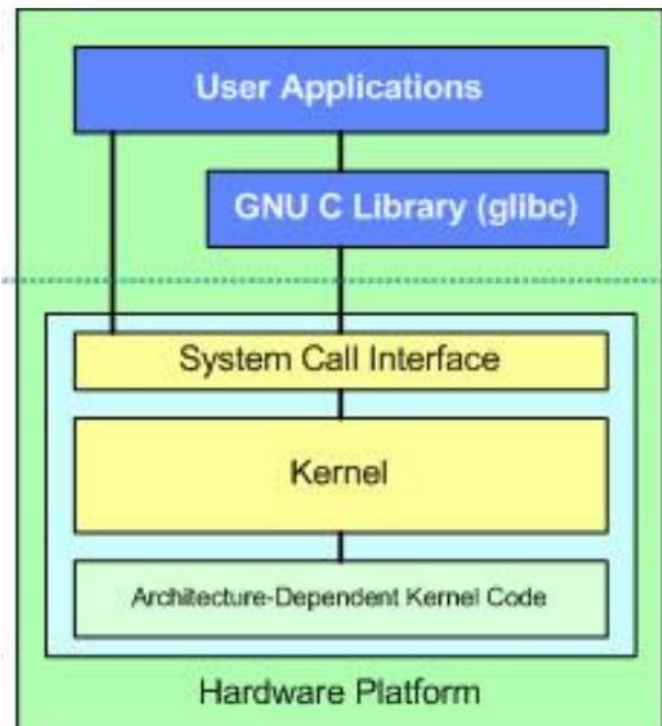
- Subsystems must agree on a compromise data model → lower performance
- Data evolution: adopting a new data model is difficult and expensive:
 - it must be applied to the entire repository,
 - all subsystems must be updated
- Different subsystems may have different requirements for backup, security, etc. that are not supported
- It is difficult to efficiently distribute the repository across multiple machines (while continuing to view it as logically centralized): problems with redundancy and data consistency.

Layered Architecture

- Sub-systems are organized into layers
- Each layer:
 - uses the services of the layer below
 - provides services to layer above through well-defined interfaces
- The terms “tier” and “layer” are interchangeable, for most people



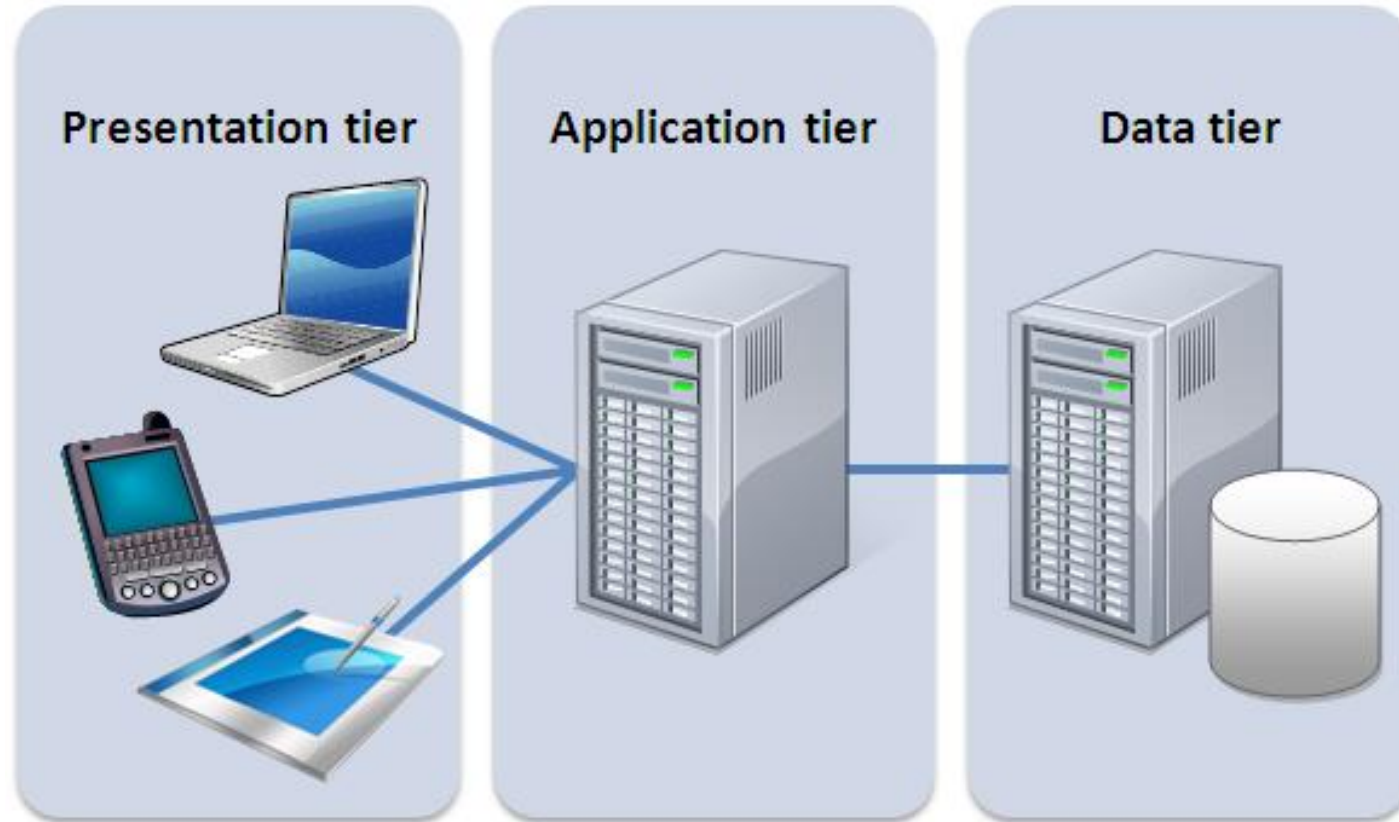
Layered Architecture Examples



Layered Architecture

- + abstraction of concerns
- + isolation of lower and higher levels from one another (i.e. decoupled)
- + reusability
- - overhead of going through different layers
- - complexity

3-tier Architectures



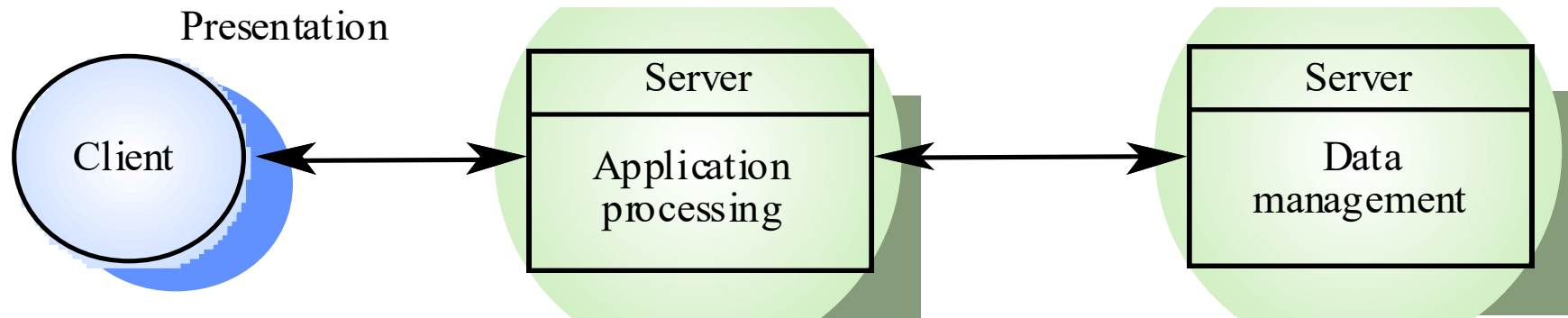
Three-tier Architecture - I

- Introduced in the 90s.
- Business logic handled explicitly:
 - Level 1: data management (communication with DBMS, XML files, etc.)
 - Level 2: business logic (data processing, etc.)
 - Level 3: user interface (data presentation, services)
- Each level has its own design objectives and constraints.
- No level makes assumptions about the structure or implementation of the others:
 - Level 2 makes no assumptions about data representation or user interface implementation.
 - Level 3 makes no assumptions about how business logic operates.

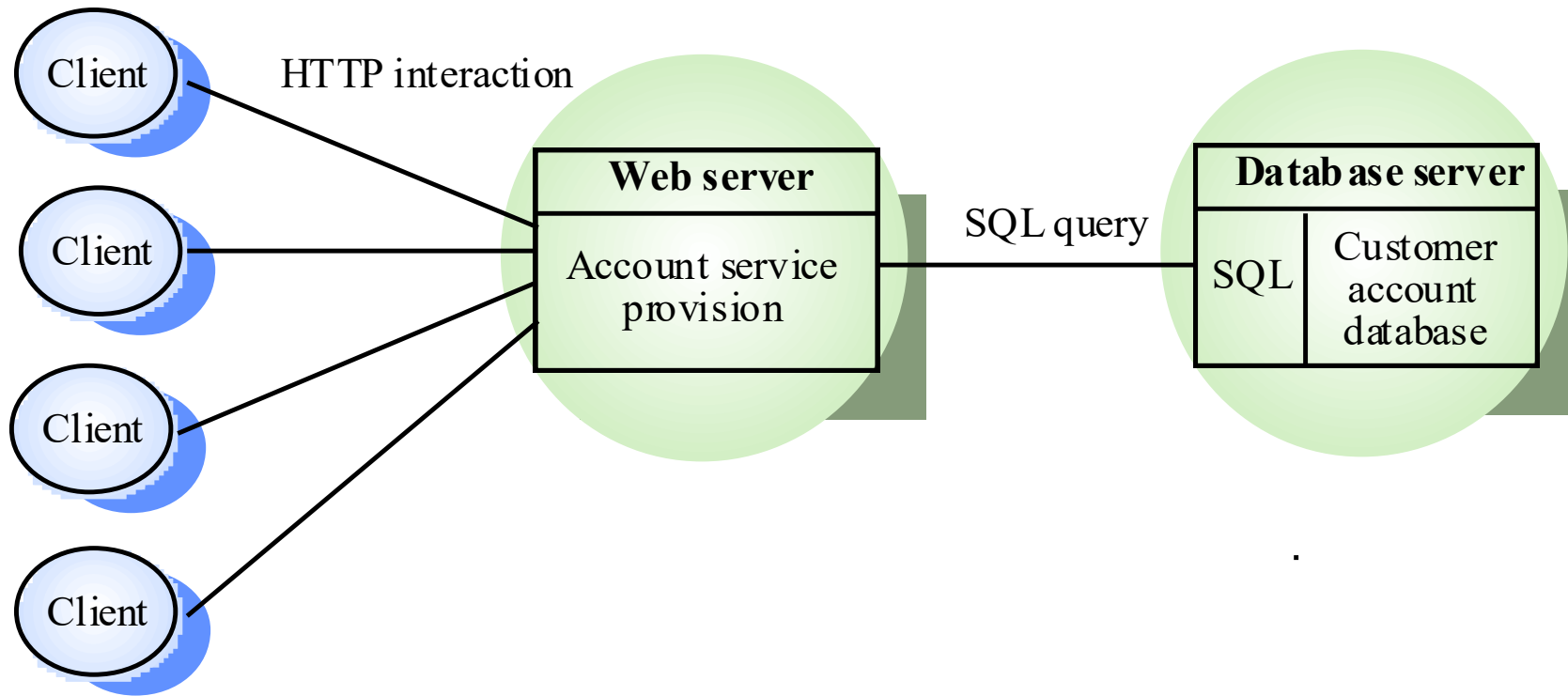
Three-tier Architecture - II

- There is no direct communication between level 1 and level 3
 - The user interface does not receive or directly enter data into the data management layer
 - All information passing in both directions is filtered by business logic
- The layers operate without assuming that they are part of a specific application
 - $\Rightarrow$ applications are seen as collections of cooperating components
 - $\Rightarrow$ each component can simultaneously be part of different applications (e.g., database, or complex object configuration logic component)

A 3-Tier Client-Server Architecture



An Internet Banking System



Benefits of three-tier architectures

- Flexibility and modifiability of systems made up of separate components:
 - components that can be used in different systems
 - modification of one component does not affect the rest of the system (unless there are changes to the API)
 - more focused bug hunting (separation and isolation of system functionality)
 - adding functionality to the application involves extending only the components involved (or adding new components)

Benefits of three-tier architectures

- Interconnectivity
 - Component APIs overcome the problem of client-server model adapters → N different interfaces can be connected to the same service, etc.
 - Facilitated access to common data by different applications (use of the same data manager by different business logics)
- Management of distributed systems
 - Business logic of distributed applications (e.g., information systems with some replicated servers and remote clients) can be updated without requiring client updates

Drawbacks of three-tier architectures

- Application size and efficiency
 - Heavy use of network communication $\Rightarrow$ service latency
 - Communication between components requires the use of SW libraries for information exchange $\Rightarrow$ bulky code
- Problems integrating legacy software
 - Many companies use old software (based on a monolithic model) to manage their data $\Rightarrow$
 - difficult to apply the three-tier model to new applications
 - introduction of adapters to interface with legacy SW

Multi-tier architectures

- Evolution of 3-tier, on N levels
- They allow for different configurations.
- Fundamental elements:
 - User interface (UI)
 - manages interaction with the user
 - can be a web browser, mobile app, graphical interface, etc.
 - Presentation logic
 - defines what the UI presents and how to manage user requests
 - Business logic
 - manages the application's business rules
 - Infrastructure services
 - provide additional functionality to the application components (messaging, transaction support, etc.)
 - Data layer:
 - application data level

Service-oriented Architecture

- Services in a service-oriented architecture are stateless components, which means that they can be replicated and can migrate from one computer to another.
- Many servers may be involved in providing services
- A service-oriented architecture is usually easier to scale as demand increases and is resilient to failure.

Service-oriented Architecture

